
System Analysis and Design

IT-353

Project

DEADLINE:

PHASE 1 (MID PROJECT PROGRESS REPORT) 26/07/2026 @ 23:59

PHASE 2 (FINAL PROJECT REPORT) 9/8/2026 @ 23:59

[Total Mark is 20]

Student Details:

CRN:

Name: fatma alsaokhan

ID: s200095074

Name:

ID:

Name:

ID:

Name:

ID:

Name:

ID:

Instructions:

- You must submit two separate copies (**one Word file and one PDF file**) using the Assignment Template on Blackboard via the allocated folder. These files **must not be in compressed format**.
- It is your responsibility to check and make sure that you have uploaded both the correct files.
- Zero mark will be given if you try to bypass the SafeAssign (e.g. misspell words, remove spaces between words, hide characters, use different character sets, **convert text into image** or languages other than English or any kind of manipulation).
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented. This includes filling your information on the cover page.
- You must use this template, failing which will result in zero mark.
- You **MUST** show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- Late submission will result in ZERO mark.
- The work should be your own, copying from students or other resources will result in ZERO mark.
- Use **Times New Roman** font for all your answers.

Project submission guidelines and instructions

1- Mid project Progress Report:

Students are required to submit a short mid progress report, which must include:

- a. Work completed this period
- b. One key decision has been made during this period
- c. One problem encountered
- d. Plan to do next.
- e. Each group member input/participation.
- f. A link to each group member GitHub repository

2- Version History / Commit Log Requirements:

Each group member must complete and submit project history/timestamp using a live, editable GitHub link. The repository must show commit history reflecting incremental progress. commit regularly (at minimum once per week), with clear, descriptive commit messages.

A separate section in the report must be added and named "Project Artifacts" to include each student's name and a link to his/her GitHub repository (with commit history).

What NOT to do: Students must not write their report elsewhere and paste the final version in at the last minute. They also should not make a single commit containing your entire files. Both are treated as red flags regardless of the quality of the final work.

3- Written Reflection (End of Project Report):

The project report should include a Reflection section (300–500 words) as the last section (one section for each student). This section must be written in students' own words and should address some of the following:

- a. Describe one specific challenge or obstacle you encountered during this project and explain in detail how you identified and resolved it.
- b. Explain why you chose your specific approach/method/model over at least one alternative you considered, and what tradeoffs informed that decision.
- c. If you used any AI tools during this project (e.g., ChatGPT, Claude, Copilot), disclose which tool(s), for what specific purpose (e.g., debugging, grammar checking, brainstorming), and how you verified or modified the output.
- d. What would you do differently if you were to redo this project with more time or resources?

Important Notes

- AI use: Students may use AI tools (ChatGPT, Claude, Copilot, Gemini, etc.) only for brainstorming approaches, debugging small errors, or proofreading text. **AI must not be used to generate analysis, write code logic, or produce substantial portions of report or codebase. Any permitted AI use must be disclosed in Reflection section. Undisclosed AI use, or AI use beyond this scope (disclosed or not), will result in a zero grade for the entire project.**
- Any submission lacking a valid, inspectable version/commit history, or showing evidence of AI generation or plagiarism, will receive a zero grade for the entire project.
- Failure to submit mid progress reports as required, or submission of reports found to be fabricated, copied, or AI-generated, will result in a zero grade for the entire project.

Project Rubric

Component	Weight	Note
Final report/deliverable	85%	Core assessment of the work itself.
Mid Progress report	10%	Incentivizes engagement across the timeline, not just at the end. Absence blocks credit for the whole project.
Version/commit history	Pass/fail	Not a grade component. Absence blocks credit for the whole project.
Reflection section	5%	Rewards demonstrated understanding and honest disclosure.

*Learning
Outcome(s):*

*Illustrate most
common analysis
and design
techniques in the
form of a group
project.*

Scenario

The Saudi Electronic University (SEU) currently organizes a wide variety of extracurricular activities, including workshops, seminars, competitions, volunteering opportunities, social events, career development sessions, and sports activities. However, information about these activities is scattered across emails, social media platforms, and announcements, making it difficult for students to discover upcoming events and register for them. Activity organizers also face challenges in managing registrations, tracking attendance, monitoring participation limits, and preparing reports.

To address these issues, SEU plans to develop a Student Activities Management System (SAMS) that will provide a centralized platform for managing all student activities. The proposed system should allow students to browse available activities by category, campus, or date, view detailed information about each activity, reserve a seat, cancel a reservation before the registration deadline, receive notifications and reminders, and record their attendance by scanning a QR code or using another electronic attendance method at the event. Students should also be able to view their personal activity history throughout their studies, including the number of activities attended, participation hours, certificates earned, and achievements. Upon graduation, the system should automatically generate a comprehensive Student Activity Report (Activity Transcript) summarizing all extracurricular activities completed during the student's academic journey.

Activity organizers should be able to create, edit, publish, and cancel activities, define participant limits, registration periods,

locations (physical or virtual), attendance methods, and eligibility requirements. Organizers should also manage participant lists, approve registrations when required, record attendance, issue participation certificates, and generate reports on attendance rates and student engagement.

The Student Affairs Department should oversee the entire system by managing activity categories, approving activities submitted by organizers, monitoring participation statistics across all university branches, generating institutional reports, and ensuring that each student's activity transcript is accurate and complete before graduation.

The system should also support notifications for registration confirmations, activity reminders, cancellations, and waitlist updates. If an activity reaches its maximum capacity, additional students should automatically be placed on a waiting list and promoted when vacancies become available.

The university expects the new system to improve student engagement, simplify activity management, eliminate manual attendance tracking, and provide accurate records of students' extracurricular involvement throughout their university studies.

Based on the above scenario, complete the following tasks.

1. Introduction and scope of order

The Student Activities Management System (SAMS) aims to standardize the advertising of extracurricular activities and manage their life cycle from creation, accreditation, and publication to registration, attendance, certification, and reporting. The system addresses the dispersion of information between email, communication and advertising, and provides the student with a central channel for review, registration, and follow-up, as well as providing the organizer and the Student Affairs Department with institutional operating and control tools.

1.1 System Objectives

- Increase student engagement by making activities available in a unified platform that can be searched and filtered.
- Facilitate enrollment, capacity, queue, and approvals management.
- Replacing manual attendance with an electronic mechanism such as a QR Code with an audit record.
- Creating an accurate record of the student's hours, certificates, and achievements throughout the study.
- Automatically generate an Activity Transcript upon graduation after the record is validated.
- Providing operational and institutional reports at the activity, branch and university levels.

2.1 Parties and Scope of Liability

Party	Key Responsibilities
Student	Browse activities, register and cancel, receive notifications, check attendance, view history and testimonials, and activity detection
Activity Organizer	,Create, modify, publish and cancel the activity, adjust capacity, duration venue and eligibility, manage participants, attendance, testimonials and reports
Student Affairs Administrator	,Manage rankings, approve activities, follow up on branch statistics corporate reports, and pre-graduation verification
Graduation System	Request to create or verify the activity statement at graduation procedures
Notification Service	Deliver registration confirmations, reminders, cancellations, and queue updates.

2. First Task: Gantt Chart of the Project

Schematic Assumption: The duration of the project is 16 school weeks, and the start of the first week is measured without assuming an unstated calendar date. The numbering of the dependencies in the table matches the order of the activities.

Accreditations	The Beginning	Duration	Activity	Number
—	Week 1	One week	Requirements Collection	1
1	Week 1	One week	System Analysis	2
2	Week 1	One week	Choosing a prototype method	3
2	Week 1	One week	Architectural System Design	4
3	Week 2	One week	Design of Primary Screens	5
2	Week 2	One week	Context and Level-0 DFD Setup	6
6	Week 2	One week	Level-1 DFD Charts	7
2	Week 2	One week	Use Case Model	8
8	Week 2	One week	Activity Diagrams	9
4	Week 2	One week	Class Diagram	10
10	Week 2	One week	Database Design and Normalization	11
11	Week 2	One week	Review and Integration	12
12	Week 2	One week	Preparation of the Final Report	13
13	Week 2	One week	Review & Delivery	14

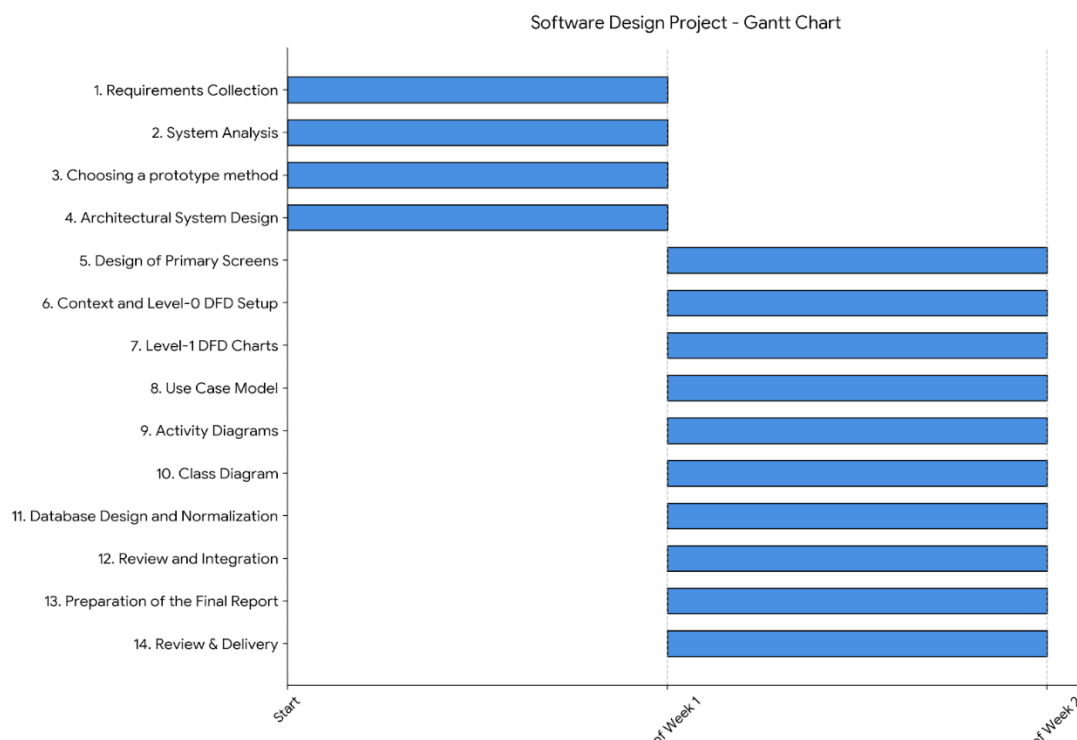


Figure 1: Proposed Gantt Chart for the SAMS Project over a 16-Week Period

2.1 Dependencies logic

- Gathering requirements precedes analysis because models and processes must be built on a clear scale.
- The design of the screens depends on the choice of prototype style and architectural design.
- Level-1 charts rely on the completeness of the Context Diagram and Level-0 so that the boundaries of operations and data stores remain consistent.
- Class Diagram takes advantage of processes and use cases, and then becomes a direct input to database design.
- Integration and final review begin only after all forms have been completed; the report is prepared and delivered.

3. Task Two: Prototype

3.1 Method chosen: Evolutionary Prototyping

Evolutionary Prototyping was chosen because SAMS is a multi-party system and contains operational rules that may become increasingly clear, such as eligibility requirements, approval of enrollment, student transition from the waitlist, and proof-of-attendance controls. This approach allows for the construction of a simplified prototype and then its development on sequential courses based on feedback from students, organizers, and student affairs management, while retaining the proven parts rather than disposing of them entirely.

3.2 Why is it best suited for the project?

- Different usage needs exist between the student, the organizer, and the administration, and they can be tested early.
- The ease of browsing, recording, and scanning QR needs actual user experiences before final installation.
- Wait, capacity, and approvals rules are adjustable when new real-world cases are detected.
- The model can be gradually transformed into a workable product with improved design, security, and performance in each version.
- It reduces the risk of building an entire system and then discovering that the registration or attendance steps are not suitable for users.

3.3 Proposed evolution cycle

Course	Scope	Verification Output
Session 1	Browse activities, filtering, and details	Test the activity's findability and understanding of its information.
Session 2	Registration, Cancellation and Queue	Test grammar, messages, and fillings.
Session 3	QR , attendance and certifications	Accuracy testing, speed of proof of attendance, and exception handling.
Session 4	Reports and Activity Disclosure upon Graduation	Assembly validation and institutional verification.

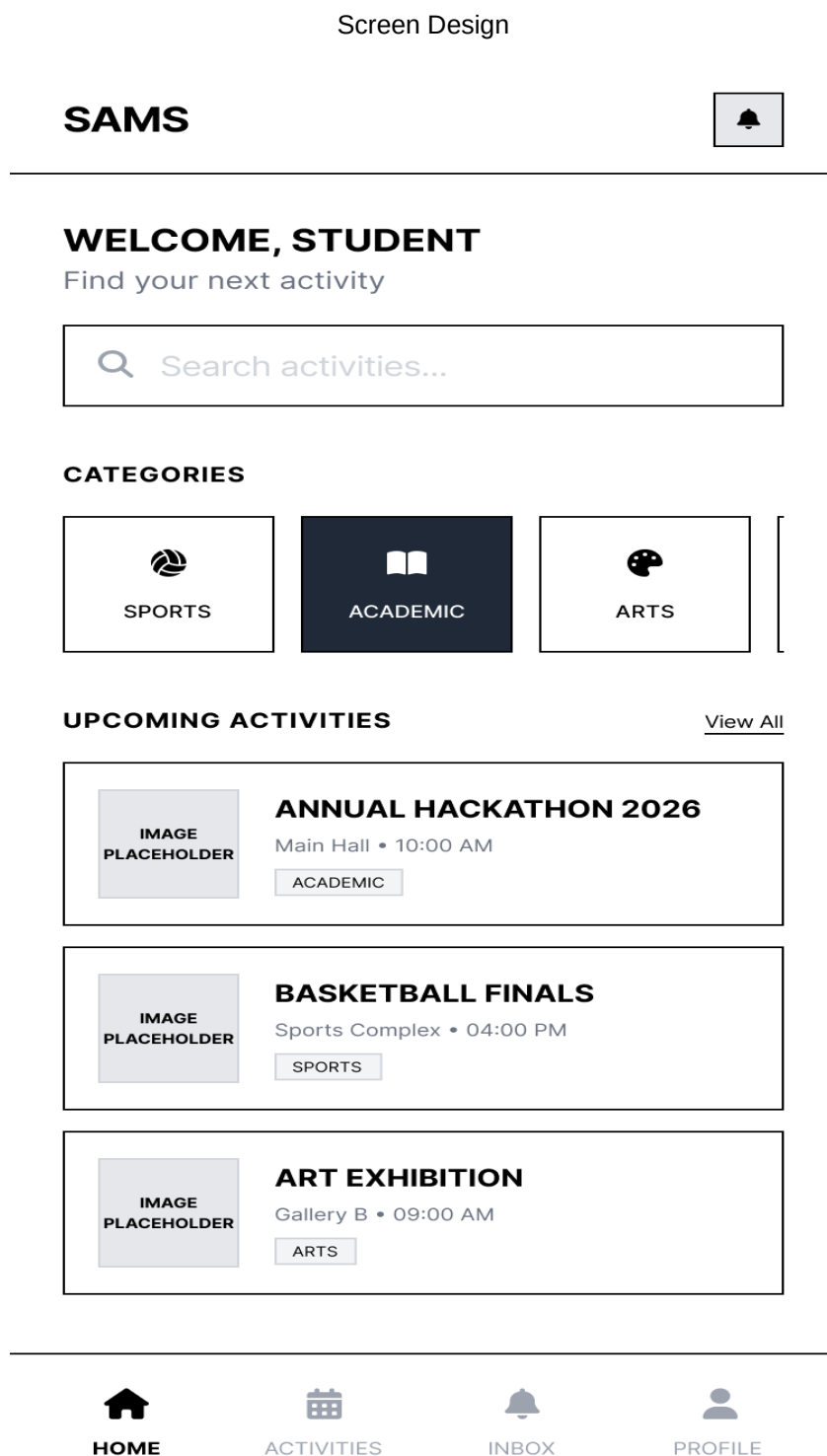


Figure 2: Low-resolution homepage model
Page

Screen Design

SAMS



BROWSE ACTIVITIES

Search by name or club...

ALL CATEGORIES



THIS MONTH



24 ACTIVITIES FOUND

SORT BY: DATE

ROBOTICS WORKSHOP 2026

ACADEMIC

Oct 12, 2026 Lab 402

AVAILABILITY
12 / 30 SEATS

REGISTER

PHOTOGRAPHY WALK

ARTS

Oct 15, 2026 West Campus

AVAILABILITY
WAITLIST ONLY

WAITLIST

CHESS CHAMPIONSHIP

SPORTS

Oct 20, 2026 Student Union

AVAILABILITY
45 SEATS LEFT

REGISTER

JAZZ NIGHT SESSION

MUSIC

Oct 22, 2026 Music Hall

AVAILABILITY
10 SEATS LEFT

REGISTER



HOME



ACTIVITIES



INBOX



PROFILE

Figure 3: Low-resolution model of the activity browsing screen

ATTENDANCE

SCAN QR CODE

OR ENTER CODE MANUALLY

Activity Code

CHECK-IN

RECENT ATTENDANCE

[View History](#)

Robotics Workshop 2026

Oct 12, 2026 • 14:15

VERIFIED

Photography Walk

Oct 10, 2026 • 09:05

PENDING

Home

Activities

Scan

Inbox

Profile

Figure 4: Low-resolution model of recording and attendance screen

3.4 Usability Notes

- Clearly show registration status, capacity, and end of the period before confirming the booking.
- Use a uniform label for statuses: Draft, Pending, Published, Closed, Canceled, Completed.
- Provide specific error messages when you are ineligible or close registration instead of a public message.
- Separate the attendee's QR from the booking code or number, with a time window specified for verification.
- Support access from the phone because registration and attendance often happen on the go.

4. Task Three: Data Flow Charts (DFDs)

The following charts use third-parties, processes, and data stores while maintaining the principle of balancing: Inflows and outflows in the Context Diagram are detailed and consistent in Level-0 and then in Level-1.

4.1 Context Diagram

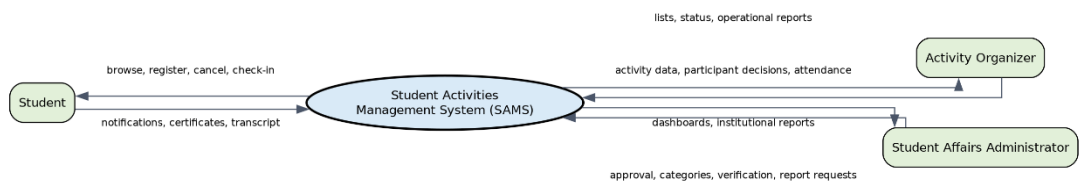


Figure 5: Context Diagram for SAMS

The system appears as a single process that exchanges data with the student, the activity organizer, and the student affairs department. The notification service and the graduation system are represented in the use case model as they are technical integrations, while in the Context Diagram it focuses on the primary human actors sending and receiving business data.

4.2 Level-0 DFD

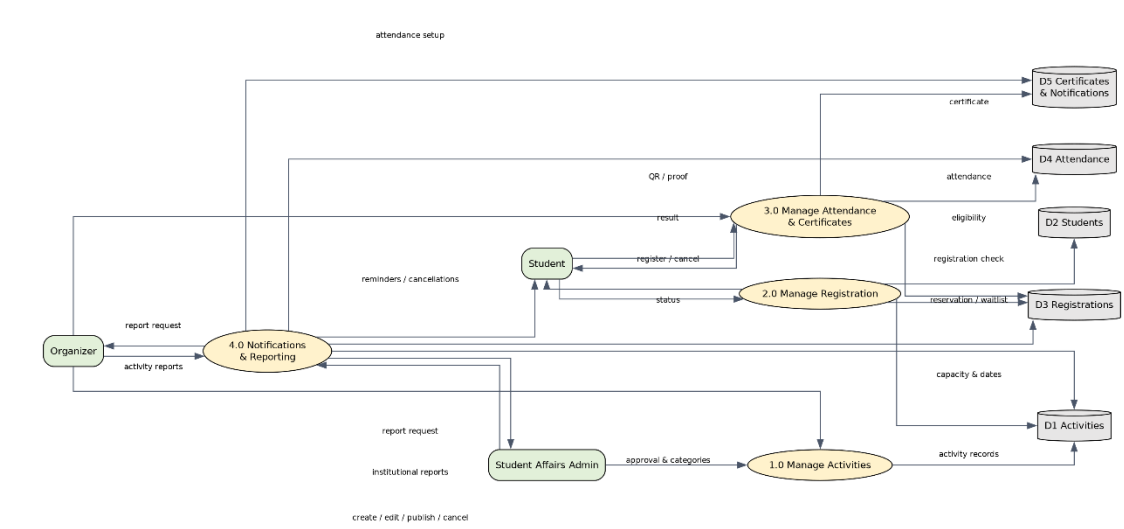


Figure 6: Level-0 DFD and the Four Main Processes

Operation Number	Operation Name	Description	Core Data Stores
1.0	Activity Management	Create, verify, approve, publish, or cancel the .activity	D1 Activities
2.0	Registration Management	,Eligibility and capacity check, reservation .cancellation, and waitlist	D1, D2, D3
3.0	Attendance and Certification Management	,Preparing the means of attendance verification, registration, and issuance of .certificates	D3, D4, D5
4.0	Notifications and Reports	,Send messages and compile activity, branch .and university reports	D1, D3, D4, D5

4.3 Level-1 Activity Management :

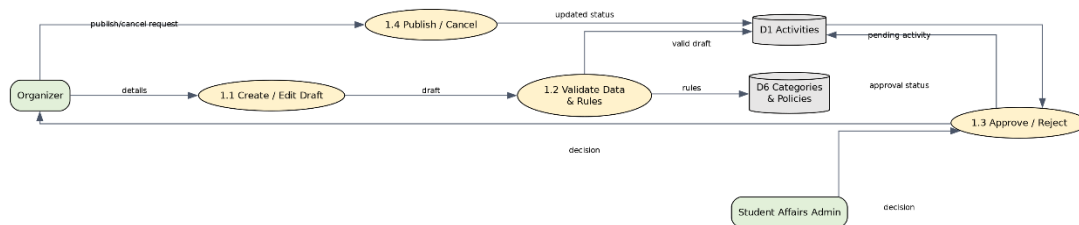


Figure 7: Level-1 for Process 1.0 Activity Management

- 1.1 Create/edit a draft: Collects title, description, classification, branch, time, capacity, location, and rules.

- 1.2 Verification of data and rules: Publication is prohibited in the absence of mandatory fields or date conflicts.
- 3.1 Adoption/Rejection: Management records the decision and comment and returns the status to the regulator.
- 1.4 Post/Cancel: Changes the status of the activity and triggers relevant notifications.

4.4 Level-1 Student Registration :

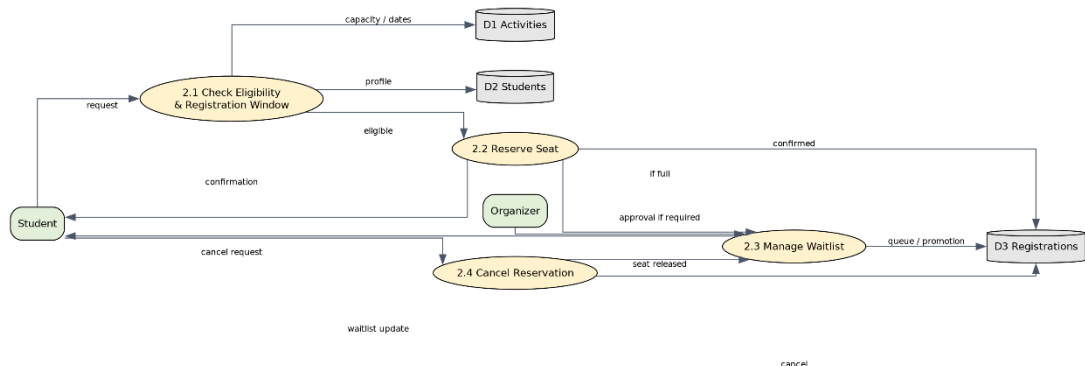


Figure 8: Level-1 for Process 2.0 Registration Management

- 2.1 Examination of the registration period, eligibility and duplicate registration.
- 2.2 Seat reservation when capacity is available, with pending approval status if the activity requires approval.
- 2.3 Manage the queue in a fixed order according to the time of joining, and automatically transfer the first student when a seat is available.
- 2.4 Cancel the booking before the end of registration and launch the upgrade process from the waiting list.

4.5 Level-1 Attendance Management :

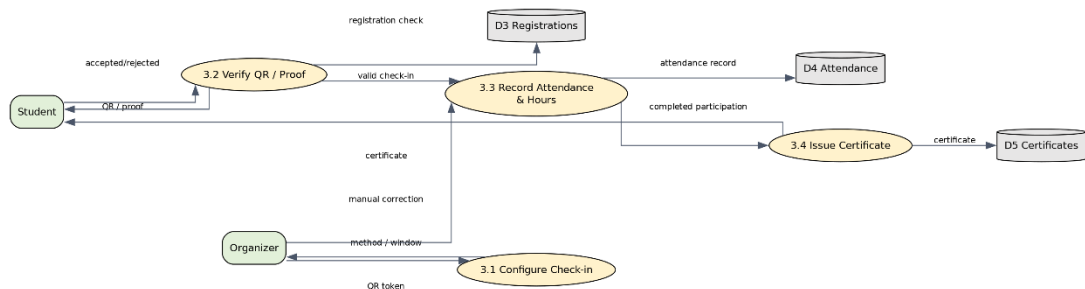


Figure 9: Level-1 of Process 3.0 Attendance Management and Certifications

- 3.1 The Organizer specifies the means of attendance and its time window and generates the session code.
- 3.2 The system verifies the validity of the QR and the confirmed student registration and non-repetition.

- 3.3 Attendance time, method, status and hours of participation shall be saved, with authorized manual adjustment when required.
- 3.4 Issue a certificate associated with completed registration and certified attendance.

4.6 Brief Data Dictionary

Data Element	Content
Activity Data	,Title, Description, Category, Branch, Date & Time, Style, Location/Link .Capacity, Registration Period, Hours, Status
Registration Request	student_id ,activity_id time of order, approval status, booking or waiting , .status
Attendance Proof	registration_id ,Session Code, Scan Time, Attendance Method , .Verification Result
Activity Record	,Activities attended by the student, hours, certificates, achievements .status of verification
Report Request	Time Range, Branch, Classification, Activity, Attendance and Participation Indicators.

5. Task Four: Use Case Model

5.1 All Actors

Actor	Type	Floor
Student	Basic	Searches, records, cancels, confirms attendance, and .reviews the record and certificates
Activity Organizer	Basic	,Manages activity, participants, attendance .testimonials, and operational reports
Student Affairs Administrator	Basic	Manages rankings, accreditation, statistics, and pre-graduation verification
Graduation System	External secondary	Request or verify theActivity Transcript within the .Graduation Procedure
Notification Service	External secondary	Performs sending mail, messages, or notification .within the app

5.2 Use Case Diagram

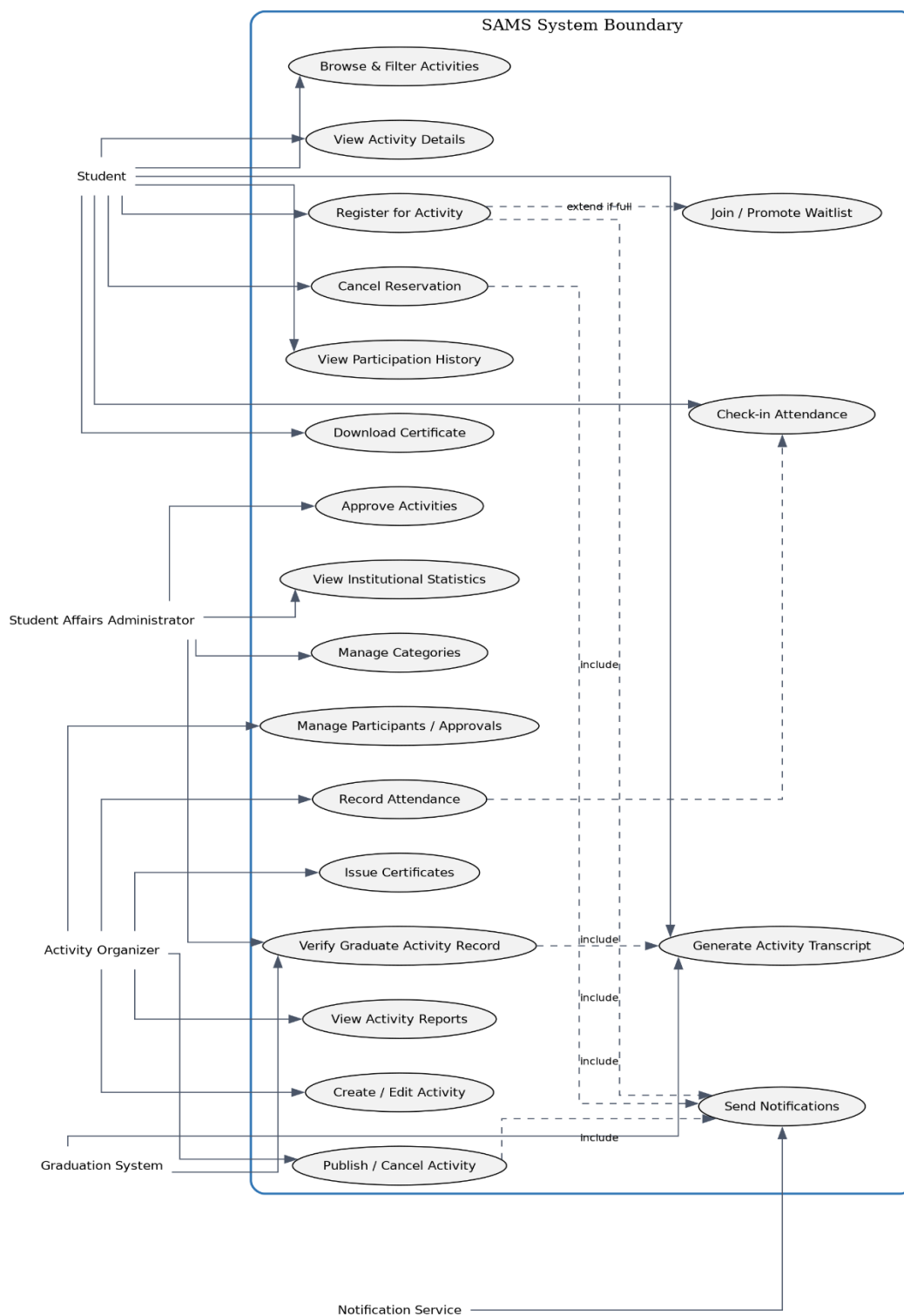


Figure 10: Complete SAMS Use Case Diagram

5.3 document Use Case 1: Register for Activity

The field	Documentation
Name	Register for Activity — Register for an Activity
Objective	Reserve a seat for a student in a qualifying and available activity, or add it to the waiting .list when full
Actor	Student; the Activity Organizer may intervene when approval is required .
Prerequisites	The student is logged in and his account is active; the activity is published; the registration period is open.
Catalyst	The student presses the button to book a seat.
Main scenario	1) The order displays the details. 2) The student requests registration. 3) The system checks the eligibility, non-repeatability and period. 4) Check the capacity. 5) Creates a Registration with a status of Confirmed or Pending Approval. 6) Updates the number and sends the confirmation.
Alternative scenario	A1: Not eligible or closed period: The application is rejected with the reason. A2: Activity is full: Creates a waitlisted and saves the queue. A3: Needs Approval: Pending Approval remains until the regulator's decision. A4: Previous Recording: Prevents repetition.
The end result	.Correct saved registration, sender notification, capacity consistency or queue position

5.4 Use Case 2: Cancel Reservation

The field	Documentation
Name	Cancel Reservation
Objective	Enable the student to cancel his/her reservation within the allowed time and make the .seat available to the first waiting student
Actor	Student
Prerequisites	The student is logged in, has a revocable registration, the cancellation /registration period has not ended.
Catalyst	The student chooses to cancel the reservation and confirms the decision.
Main scenario	1) The system retrieves the registration. 2) Checks for cancellation. 3) Changes the status to Cancelled and records the time. 4) Releases the seat. 5) The first eligible student is selected in the queue and transferred automatically. 6) Sends two notifications.
Alternative scenario	A1: Timed out: Refuses to cancel and shows the reason. A2: No waiting student: The seat remains available. A3: The first student who is no longer eligible: exceeds it according to a documented policy and takes the next test with the audit record kept.
The end result	.Cancel Saved ; Capacity Updated; Queue Updated; Notifications Recorded

5.5 Use Case 3: Create Activity

The field	Documentation
Name	Create Activity — Create Activity
Objective	.Create a completed activity draft and submit it for approval before publishing
Actor	Activity Organizer , Student Affairs Administrator.
Prerequisites	The organizer is logged in and authorized; active classifications and branches are available.
Catalyst	The organizer chooses to create a new activity.
Main scenario	1) The organizer enters the basic data. 2) Determines capacity, enrollment period, style, location, attendance, and eligibility. 3) The system checks the fields and dates. 4) Saves the draft. 5) Sent by the organizer for approval. 6) The system records the Pending Approval status and notifies the administration.
Alternative scenario	A1: Incomplete data or invalid dates: identifies errors and does not allow transmission. A2: Location or Time Conflict: Displays a warning or denies depending on the policy. A3: Admin Rejection: Draft is re-submitted with comment for modification and resubmission.
The end result	Activity is saved with a unique ID and a valid status, with a traceable credential record.

6. Task Five: UML Activity Diagrams

6.1 Student Registration in the Activity

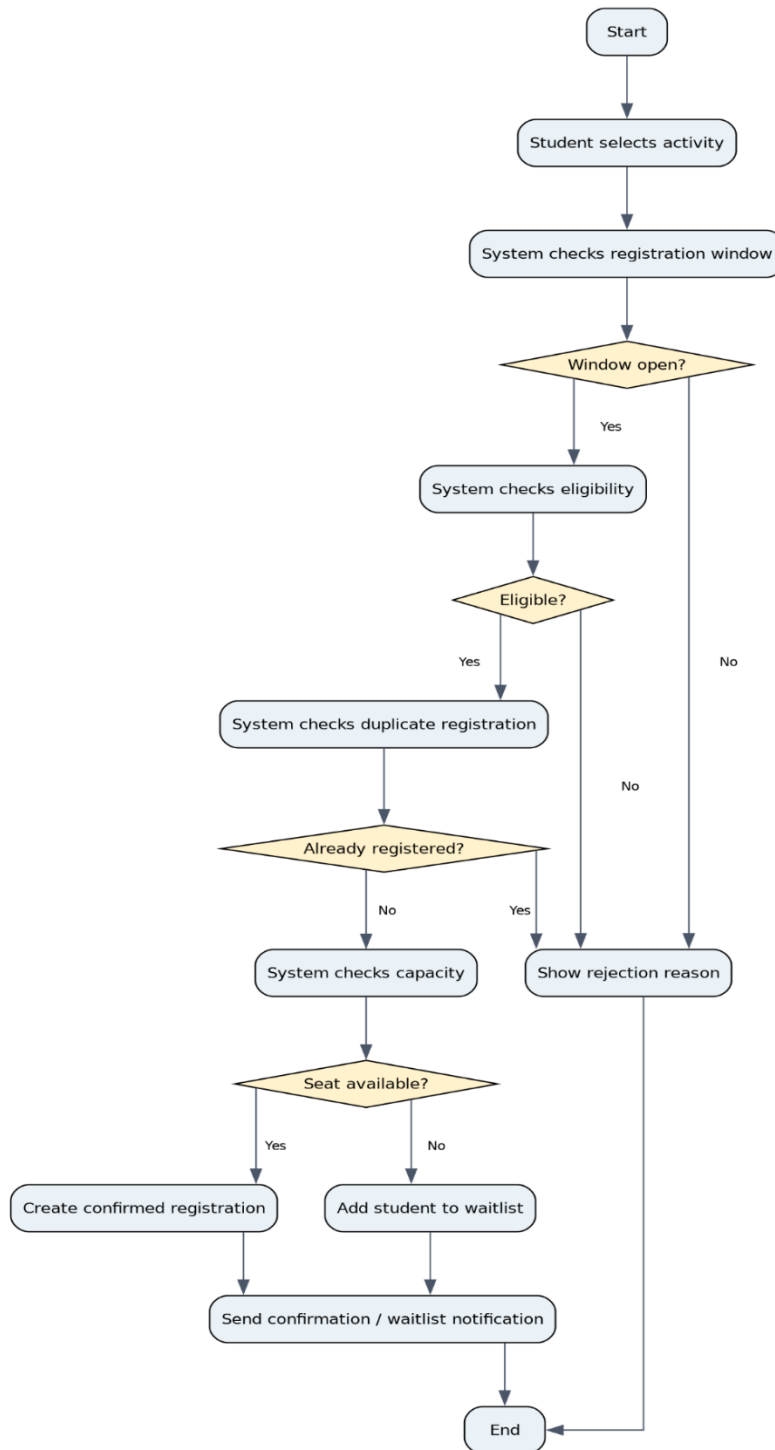


Figure 11: Activity Diagram for Student Registration in the Activity

The chart shows the decision points that prevent incorrect registration: enrollment window, eligibility, non-recurrence, and capacity availability. If a seat is not available, the flow moves to the queue instead of rejecting the student directly.

6.2 Attendance Registration withQR Code

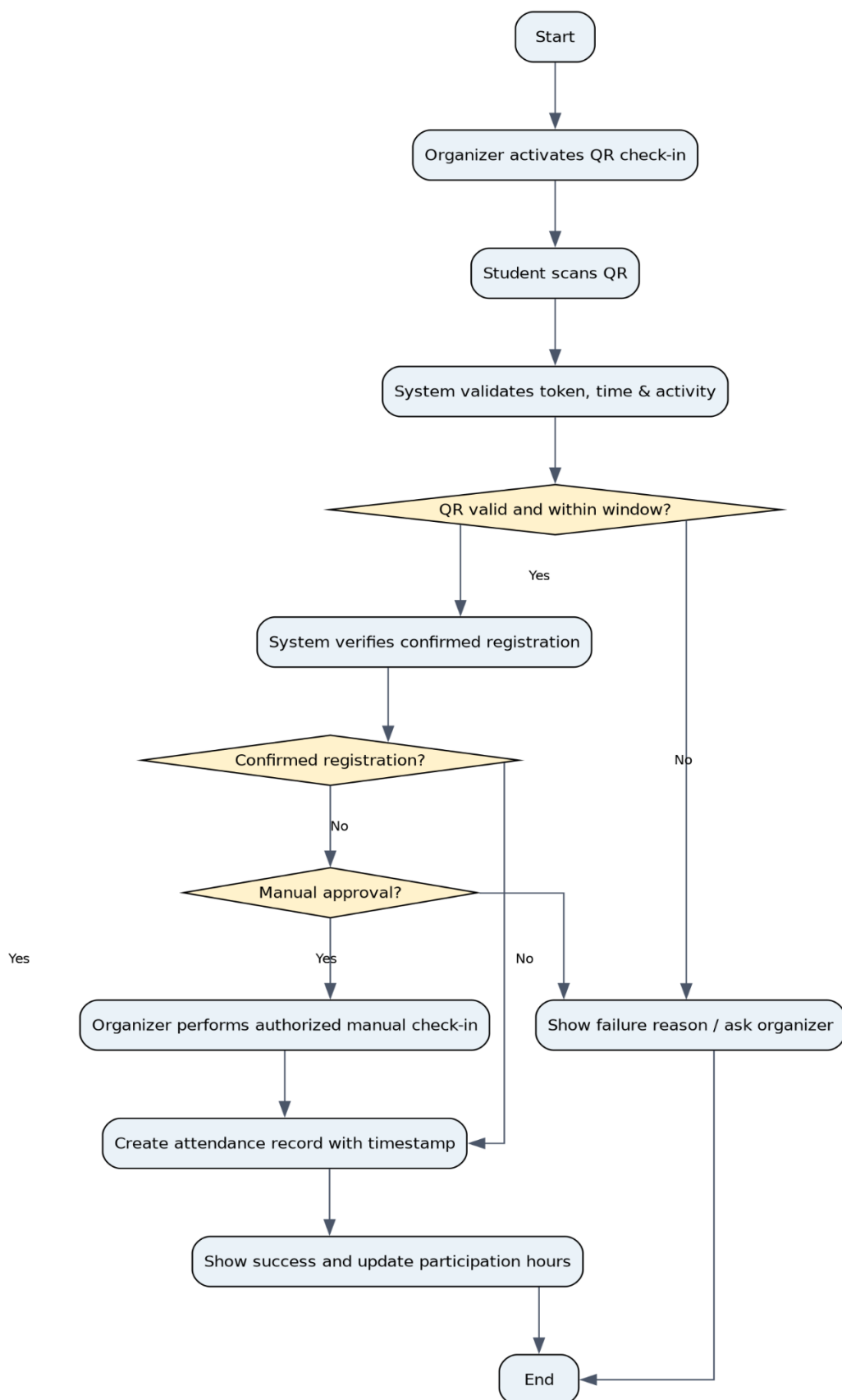


Figure 12: Activity Diagram for Attendance Recording by QR Code

The validity of the code and time window is verified and the attendees are linked to a confirmed registration. The swing route provides an authorized manual procedure for the regulator in exceptional cases, while the recording remains in the audit log.

Page

:Task Six .UML Class Diagram

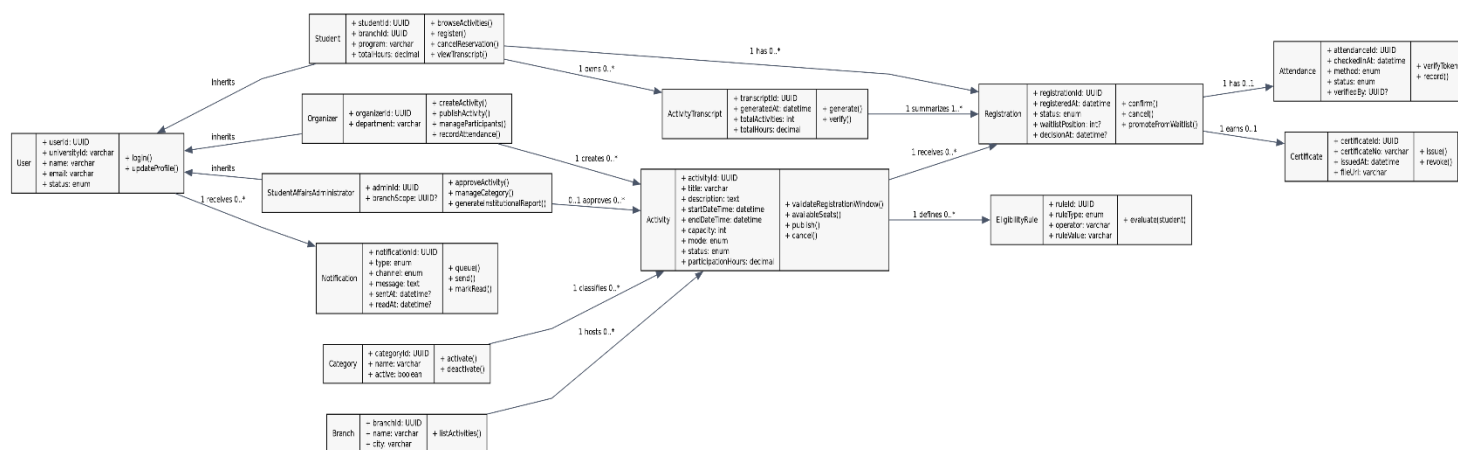


Figure 13: Class Diagram for SAMS

7.1Classes and relationships explained

Class	Responsibility	Main Relationships
User	Stores shared account information and provides authentication for all system users.	Superclass of Student , Organizer , and Student Affairs Administrator .
Student	Registers for activities, cancels registrations, records attendance, and views activity history.	One Student can have 0..* Registrations and 0..* Notifications .
Organizer	Creates and manages activities, participants, and attendance records.	One Organizer can manage 0..* Activities .
Activity	Represents an activity with its details, rules, capacity, and current status.	Associated with one Category , one Branch , multiple Registrations , and multiple Eligibility Rules .

Registration	Links a student to an activity and stores the registration status such as Confirmed, Pending, Waitlisted, or Cancelled.	Each Registration may have 0..1 Attendance and 0..1 Certificate .
Attendance	Records attendance details including the attendance time, method, and verification status.	Each Attendance record is linked to exactly one Registration .
Certificate	Represents the certificate issued after successful participation in an activity.	Each Certificate is linked to one eligible Registration .
Notification	Stores messages sent to users and may be related to a specific activity.	Each User can receive 0..* Notifications .
ActivityTranscript	Represents a summary of the student's completed activities, attendance, certificates, and participation hours.	Generated from completed Registrations , Attendance , and related activity records.

7.2 Limitations of the Domain Model

- Confirmed registrations may not exceed the capacity of the activity.
- A unique restriction on (student, activity) prevents effective duplicate enrollment.
- Attendance is only created for an existing registration, and a certificate is issued only after a completed and approved attendance.
- The waitlistPosition is NULL for the non-Waitlisted state, and is unique within the activity when you use it.
- User inheritance reduces the duplication of identity information, while subtables save the characteristics of roles.

8. Task Seven: Database Design

8.1 General relational model

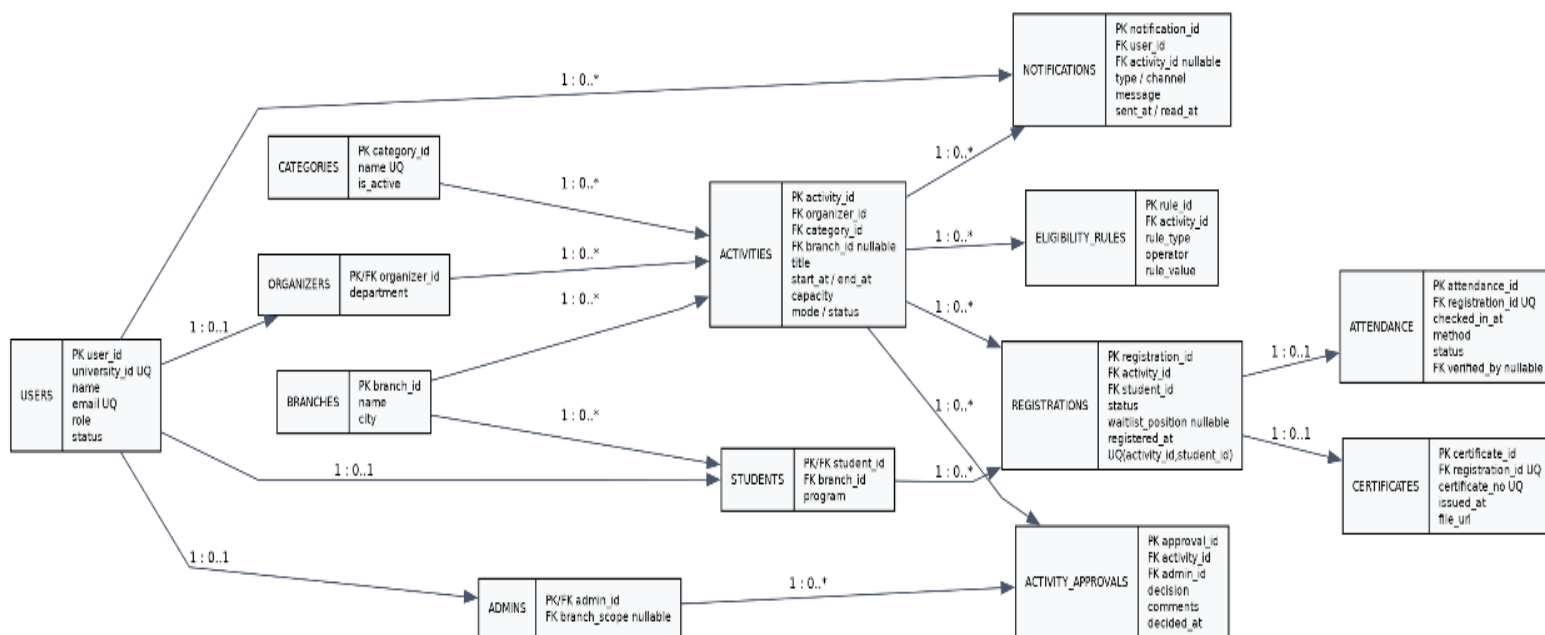


Figure 14: Proposed logical ERD for SAMS database

8.2 Tables, Keys, and Data Types

Table	Fields and Keys	Purpose
BRANCHES	branch_id UUID PK; name VARCHAR(120) UQ; city VARCHAR(100); is_active BOOLEAN	University Branches .Data
USERS	user_id UID PK; university_id Versar, 30, UK; Versace (150); Email Versace (254) UK; Roll Versher (30); Status Versace (20); created_at Timestamp	.Shared account of roles
STUDENTS	student_id UUID PK/FK→USERS.user_id; branch_id UUID FK→BRANCHES; program VARCHAR(120)	Characteristics of the .student
ORGANIZERS	organizer_id UUID PK/FK→USERS.user_id; department VARCHAR(150)	Characteristics of the .regulator

STUDENT_AFFAIRS_ADMINS	admin_id UUID PK/FK→USERS.user_id; branch_scope UUID NULL FK→BRANCHES	Powers of the .Administration
CATEGORIES	category_id UUID PK; name VARCHAR(100) UQ; description VARCHAR(500) NULL; is_active BOOLEAN	.Activity Classifications
ACTIVITIES	activity_id UUID PK; organizer_id UUID FK; category_id UUID FK; branch_id UUID NULL FK; title VARCHAR(200); description TEXT; start_at TIMESTAMP; end_at TIMESTAMP; registration_start TIMESTAMP; registration_end TIMESTAMP; capacity INT; mode VARCHAR(20); venue VARCHAR(250) NULL; virtual_url VARCHAR(500) NULL; attendance_method VARCHAR(30); requires_approval BOOLEAN; participation_hours DECIMAL(5,2); status VARCHAR(30); created_at TIMESTAMP	Activity and lifecycle .data
ELIGIBILITY_RULES	rule_id UUID PK; activity_id UUID FK→ACTIVITIES; rule_type VARCHAR(50); operator VARCHAR(10); rule_value VARCHAR(250)	Expandable eligibility .rules
REGISTRATIONS	registration_id NEW PK; activity_id UUID FK; student_id UUID FK; registered_at TIMESTAMP; status VARCHAR(30); waitlist_position INT NULL; decision_by UUID NULL FK→USERS; decision_at TIMESTAMP NULL; cancelled_at TIMESTAMP NULL; UQ(activity_id,student_id)	.Booking, approval .waiting, and cancellation
ATTENDANCE	attendance_id UUID PK; registration_id UUID UQ FK→REGISTRATIONS; checked_in_at TIMESTAMP; method VARCHAR(30); status VARCHAR(20); verified_by UUID NULL FK→USERS; evidence_ref VARCHAR(500) NULL	One proof of attendance .per registration
CERTIFICATES	certificate_id UUID PK; registration_id UUID UQ FK→REGISTRATIONS; certificate_no VARCHAR(80) UQ; issued_at TIMESTAMP; file_url VARCHAR(500); status VARCHAR(20)	.Certificates
NOTIFICATIONS	notification_id UUID PK; user_id UUID FK→USERS; activity_id UUID NULL FK→ACTIVITIES; type VARCHAR(40); channel VARCHAR(20); message TEXT; queued_at TIMESTAMP; sent_at TIMESTAMP NULL; read_at TIMESTAMP NULL; status VARCHAR(20)	Notifications and .reminders
ACTIVITY_APPROVALS	approval_id NEW PK; activity_id UUID FK; admin_id UUID FK; decision VARCHAR(20); comments VARCHAR(1000) NULL; decided_at TIMESTAMP	Record of accreditation .decisions
TRANSCRIPT_SNAPSHOTS	transcript_id NEW PK; student_id UUID FK; generated_at TIMESTAMP; total_activities INT; total_hours DECIMAL(7,2); verification_status VARCHAR(20); verified_by UUID NULL FK→USERS; file_url VARCHAR(500) NULL	Official copies generated upon request; details derived from recordings and attendance.

8.3 Proposed Restrictions and Indexes

Page

- CHECK (capacity > 0) CHECK (end_at > start_at) CHECK (registration_end >= registration_start).
- CHECK (participation_hours >= 0) and constraints for the values allowed in status , mode, and method.
- UNIQUE(activity_id, student_id) in REGISTRATIONS to prevent duplicate registration.
- UNIQUE(registration_id) in ATTENDANCE and CERTIFICATES to ensure a relationship of 1 to 0..1.
- Composite index of ACTIVITIES(status, start_at, category_id, branch_id) to speed up browsing.
- REGISTRATIONS INDEX(activity_id, status, waitlist_position, registered_at) to support capacity and upgrade.
- Perform a queue dialer transfer within a Transaction with a proper lock to prevent the same seat from being reserved simultaneously.
- Encrypt the session link or QR code and not store a reusable secret value in its raw form.

8.4 Normalization Implementation Up to Third Normal Form (3NF)

Stage	Problem Before Normalization	After Normalization
1NF	An activity could store multiple participants or eligibility rules in a single field.	Each field stores only one value. Participants are stored in the REGISTRATIONS table, and eligibility rules are stored in the ELIGIBILITY_RULES table.
2NF	In a combined table containing both student and activity data, some attributes depend only on the student or only on the activity.	Student and activity information are stored in separate tables. The REGISTRATIONS table is used to connect students with activities.
3NF	Activity records may contain repeated information such as branch names, category names, or organizer details.	This information is stored in separate tables and linked using foreign keys, which reduces redundancy and keeps the data consistent.

Example 3NF: In **ACTIVITIES** **activity_id** defines all activity properties. **category_name** is not stored in **ACTIVITIES** because **category_id** specifies the name within **CATEGORIES**. Also, **branch_id** the branch name and city are specified in **BRANCHES**, there is no transition dependency **activity_id** → **branch_id** → **branch_name** within the activity table.

8.5 Business Rules to Implement in the Service Tier

- DYNAMICALLY COUNT THE NUMBER OF CONFIRMED SEATS FROM REGISTRATIONS INSTEAD OF STORING AN ASYNCHRONOUS COUNTER; IF USING A PERFORMANCE OPTIMIZATION COUNTER, IT SHOULD BE UPDATED ATOMICALLY.
- When canceling: Select the first Waitlisted enrollment by **waitlist_position** and then **registered_at**, recheck eligibility, and upgrade it within a single transaction.
- Create an Activity Transcript from recordings that have Attendance approved, then count the number and hours and attach certificates—save an official snapshot only when needed.
- Maintain a record of critical approvals and adjustments for institutional review purposes.
- Hide data by role and branch and apply the principle of less validity.

9. Consistency and tracking between models

Class / Database	DFD / Use Case	Key Requirement
Activity, Category, Branch / ACTIVITIES , CATEGORIES , BRANCHES	Process 1.0: Browse Activities	Activity Browsing
Registration / REGISTRATIONS	Process 2.0: Register, Cancel, Waitlist	Registration & Queue
Attendance / ATTENDANCE	Process 3.0: Check-in	E-Attendance
Certificate / CERTIFICATES	Process 3.0: Issue Certificate	Certificates
Notification / NOTIFICATIONS	Process 4.0: Send Notifications	Notifications

Administrator / ACTIVITY_APPROVALS	Process 1.3: Approve Activities	Activity Approval
ActivityTranscript / TRANSCRIPT_SNAPSHOTS	Generate Transcript	Activity Transcript

10. Conclusion

In this project we created a solid foundation for developing the SAMS system as a centralized and scalable platform We linked the analysis and design phases and used the evolutionary prototype approach to reduce usability issues Then we converted the system requirements into data flow diagrams use cases behavioral diagrams class diagrams and the database design

Before starting the implementation we need to define the eligibility policies cancellation periods branch permissions and data retention rules We also need to test scenarios with many users such as capacity management queue handling and electronic attendance to make sure the system works efficiently